Qwen

Beyond the Black Box: A Guide to Algorithmic MIDI Harmonization with Python

Evaluation of Candidate Packages Against Research Criteria

The primary objective is to identify a suitable open-source Python package for symbolic MIDI melody harmonization that adheres to a strict set of criteria: recency (2024-2026), algorithmic methodology, support for arbitrary non-quantized monophonic melodies, high-quality output, and ease of use with minimal dependencies. An analysis of the provided materials reveals that while several promising libraries exist within the Python music technology ecosystem, none perfectly fulfill all these requirements simultaneously. The search for a single, turnkey solution from the specified recency window is unsuccessful, pointing instead toward a more strategic approach involving established toolkits. This section evaluates specific candidate packages against the user's defined constraints.

A central figure in this investigation is `AccoMontage2`, which serves as the user's reference point [3] [5]. However, a detailed examination shows it fails to meet several critical requirements. Its core paper was published in 2022, placing it outside the desired 2024-2026 release window [1] [3] [8]. More importantly, its methodology is fundamentally at odds with the user's technical specifications. `AccoMontage2` relies on a database of over 5,000 pre-existing, *quantized* chord progressions reorganized from the Niko Dataset [1]. This suggests its harmonization logic is based on pattern-matching against this fixed library rather than generating harmony from first principles using rules. Consequently, it cannot reliably process arbitrary, non-quantized melodies, a key requirement of the research goal. Furthermore, it is described as a "hybrid pathway" combining rule-based optimization with this data-driven module, blurring the line between purely algorithmic and machine learning approaches, which also falls short of the user's explicit preference [128]. While it does offer a GUI for controlling generation styles and produces multi-track accompaniments (`chord_gen.mid`, `textured_chord_gen.mid`), its operational constraints render it unsuitable [1] [124].

Other potential candidates identified in the source materials present different sets of trade-offs. `MusicLang` is a Python package designed for music notation and generation, directly aligning with the user's interest in symbolic music processing [101]. It is available via PyPI and aims to make working with symbolic music gen AI easier [53]. However, the provided documentation offers insufficient detail regarding its underlying methodology—whether it is algorithmic or ML-based—and lacks information on its dependency footprint and API complexity. Without this crucial context, it is impossible to confidently recommend `MusicLang` as a solution that meets all the nuanced requirements of the research goal.

Another relevant package is `mididarmony`, but its purpose is explicitly for harmony *analysis*, not generation [96]. It provides fast, standalone harmony analysis by comparing the chord and note relationships of a given MIDI file against a high-quality database. Therefore, it is not a tool for creating new accompaniments but rather for diagnosing existing ones, making it entirely unsuitable for the user's task.

Several other libraries were mentioned in the context of general-purpose music processing, algorithmic composition, or MIDI manipulation. These include `pycomposer`, `melodia`, `mingus`, `mididings`, and `music`. While these tools demonstrate the breadth of Python libraries available for music, the provided sources lack sufficient detail to assess their specific capabilities for monophonic melody harmonization, particularly concerning their handling of non-quantized input and their adherence to a purely algorithmic paradigm. For instance, `mingus` is described as an advanced music theory and notation package with MIDI support, but its specific features for automated harmonization are not detailed [163]. Similarly, `mididings` is a powerful MIDI router and processor, but its primary function is event transformation rather than harmonic generation [164].

The table below summarizes the evaluation of these candidate packages against the user's core requirements.

Package / Tool	Recency (2024-2026)	Algorithmic Approach	Arbitrary Non-Quantized Input	High-Quality Output	Minimal Dependencies / Ease of Use
AccoMontage2	No (2022) [3]	Hybrid (Rule-based + Data-driven) [128]	No (Relies on quantized dataset) [1]	Yes (Full accompaniment) [124]	Low (Requires external data files) [1]
MusicLang	Unknown (No release date in sources)	Unknown (Not specified)	Unknown (Not specified)	Unknown (Not specified)	Unknown (Not specified)
mididarmony	Unknown (No release date in sources)	N/A (Analysis tool only)	Yes (Analyzes any MIDI)	No (Does not generate)	Unknown (Not specified)
pycomposer	Unknown (No release date in sources)	Likely (Algorithmic Composer)	Unknown (Not specified)	Unknown (Not specified)	Unknown (Not specified)
mingus	Unknown (No release date in sources)	Likely (Theory-based)	Unknown (Not specified)	Unknown (Not specified)	Unknown (Not specified)

This evaluation highlights a significant gap in the market for a modern, actively maintained, open-source Python package that is exclusively algorithmic, handles non-quantized input, and is easy to use. The current landscape is dominated by either older systems with restrictive assumptions, ML-based approaches, or general-purpose libraries where the specific feature of interest is not well-documented. The most viable path forward, therefore, involves moving beyond searching for a single perfect package and instead leveraging a combination of foundational libraries to build a custom solution tailored to the user's precise needs. The `music21` library emerges as the most logical foundation for such an endeavor due to its active maintenance, deep integration of music theory, and flexibility, despite its own complexities [77] [181].

The Landscape of Algorithmic Music Generation

The user's explicit preference for algorithmic approaches over machine learning (ML) or hybrid models places them in a distinct niche within the broader field of computational music generation. While the provided context is rich with examples of ML-driven tools and academic research, a thorough analysis confirms that purely algorithmic solutions remain a viable and theoretically sound paradigm. Understanding this landscape is crucial for appreciating why a custom-built solution might be necessary and how it can be implemented effectively.

Recent academic conferences, such as the International Society for Music Information Retrieval (ISMIR), showcase a clear trend towards ML-based methods. Papers presented

at ISMIR 2024, 2025, and 2026 frequently discuss Transformer architectures, diffusion models, and large language models (LLMs) for tasks like chord progression generation, melody creation, and full song arrangement [30][236][254][257]. For example, Chord-Transformer uses chord progressions as high-level semantic features to guide music generation [86], while MIDI-GPT is a generative model based on the Transformer architecture for computer-assisted composition [232]. Even hybrid systems like AccoMontage2, which the user finds too dated, incorporate ML elements by learning from a large dataset of quantized chord progressions [1]. This dominance of ML is also reflected in commercial products; plugins like Harmony Engine use AI to generate complex harmonies [2], and services like Melodfy employ AI to convert audio piano recordings into MIDI [56]. This widespread adoption of ML is driven by its ability to learn complex patterns from vast datasets, often producing results that are stylistically diverse and musically coherent in ways that are difficult to program manually.

However, the user's constraint against ML is not without merit. Algorithmic composition, rooted in classical music theory and mathematical logic, offers transparency, control, and predictability. The principles of harmony—such as diatonic chord progressions, voice leading, cadences, and modal interchange—are inherently rule-based [70][72][126][204]. Many successful algorithmic systems have been built upon these foundations. For example, evolutionary algorithms have been used to generate compositions according to statistical specifications [71], and Hidden Markov Models can provide a probabilistic framework for generating chord progressions based on key transitions [49]. The very definition of harmony as the combination of chords and chord progressions is a rule-governed concept [111]. Systems like `MusicLang` aim to make these processes accessible through a Python-like programming interface [101], and general algorithmic composition frameworks like Common Music have long supported MIDI integration [246][248].

A key distinction exists between systems that *learn* harmony from data (like AccoMontage2's reliance on a quantized dataset [1]) and those that *implement* the rules of harmony. The latter aligns with the user's request. This requires a library that can represent musical concepts like keys, scales, and chords, and apply logical transformations to them. The `music21` library is a prime example of such a toolkit. It is explicitly designed as a tool for computer-aided musical analysis and computational musicology, grounded in music theory [77][160]. Its `harmony` module provides dedicated functionality for analyzing and generating harmonizations, allowing developers to apply concepts like reducing chords, identifying cadences, and checking part-writing rules [121][122]. While implementing a high-quality harmonizer with `music21` requires programming

effort to define the specific rules, it ensures the final product is genuinely algorithmic and transparent.

The challenge in building an algorithmic harmonizer lies not in the theoretical impossibility but in the practical implementation of sophisticated musical rules. Achieving a "high-quality" accompaniment means more than just matching melody notes to possible chords. It involves considerations like:

- **Voice Leading:** Ensuring smooth movement between successive chords, avoiding awkward leaps [231].
- **Chord Inversion:** Choosing appropriate inversions to create a pleasing bass line and avoid parallel fifths/octaves.
- **Rhythmic Placement:** Deciding when to change chords to complement the melody's rhythm.
- **Stylistic Appropriateness:** Applying rules that fit a specific style, such as jazz (which uses secondary dominants and modal interchange) versus classical harmony [125126].

While ML models can implicitly learn these subtleties from data, an algorithmic approach requires them to be explicitly programmed. This is a significant undertaking but one that fully satisfies the user's core requirement for a non-black-box, rule-based system. The existence of community projects like `harmonize.py` suggests that even small-scale attempts to implement these rules are being made, providing a potential starting point for a larger project [251]. Ultimately, the landscape indicates that while the user is seeking a less common approach, the necessary tools and theoretical foundations are readily available within the open-source Python ecosystem, primarily centered around powerful libraries like `music21`.

Technical Feasibility of Non-Quantized Harmonization

A critical requirement of the research goal is the ability to process arbitrary, non-quantized monophonic melodies. This specification presents a significant technical hurdle, as much of the existing infrastructure in music information retrieval (MIR) and machine learning assumes quantized, grid-aligned rhythmic data. Quantization is the process of snapping MIDI notes to a predefined rhythmic grid, which simplifies analysis but can alter the expressive timing inherent in human performance [280281]. An unquantized MIDI file contains precise timing information for each note-on and note-off

event, reflecting the original performance's micro-timing nuances [283]. Successfully harmonizing such a melody requires a system that respects this continuous-time nature of the input.

The provided context material touches upon the concept of continuous time in music, though often in the context of advanced neural network architectures. Research on Continuous-Time Recurrent Neural Networks (CTRNNs) and marked temporal point processes acknowledges the theoretical importance of modeling events in continuous time rather than discrete steps [13] [15] [17]. One paper discusses reformulating differential equations into difference equations for computational purposes, highlighting that in a computer environment, continuous time is a theoretical concept that must be approximated [11]. While these references are academic and pertain to ML models, they underscore a fundamental truth: handling arbitrary timing is computationally more complex than working with a simple grid. However, for a purely *algorithmic* system, this complexity is manageable with the right tools.

The feasibility of this task hinges almost entirely on the choice of the underlying Python MIDI library. Standard libraries designed for reading and writing raw MIDI files are inherently capable of preserving the non-quantized timing information. `pretty_midi`, for instance, is a utility library that provides classes and functions for handling MIDI data in a format that is easy to modify and extract information from, without imposing a quantized structure [97]. Similarly, `MIDIUtil` is a pure Python library specifically designed to write multi-track MIDI files from within Python programs, allowing for the direct translation of note data with its original timing offsets [189]. Other libraries like `pm-python` provide a Python wrapper for PortMidi, enabling low-level access to MIDI streams with their native timing resolution [195]. These tools serve as the essential foundation for any pipeline that must work with unquantized data. They ensure that the input melody is represented accurately, including its subtle rhythmic displacements, before any harmonic logic is applied.

The core challenge then shifts from data parsing to the design of the harmonization algorithm itself. An algorithm designed for quantized music might assume a chord changes occur on beat boundaries. An algorithm for non-quantized music must be more flexible. The harmonic logic would need to consider the melodic contour and phrasing as determined by the actual note timings, not an imposed grid. For example, a sustained melody note might justify a held chord, regardless of whether it falls exactly on a bar line. Conversely, rapid melodic passages might suggest a faster-changing accompaniment.

This is where a powerful library like `music21` becomes invaluable. `music21` is built around the concept of `Streams`, which are containers for musical objects like `Notes` and `Chords` placed in a sequence with precise timing information [120]. When a MIDI file is read into `music21`, its timing is preserved. The developer can then iterate through the melody stream, examining the duration and placement of each note. A custom algorithm could be written to analyze this stream and decide where to place chord changes based on melodic endpoints, phrase structure, or harmonic tension. For example, a simple rule could be: "Generate a new chord whenever a note in the melody has a duration longer than one sixteenth note." A more sophisticated rule could involve detecting melodic peaks or cadential figures and placing important harmonic landmarks (like I or V chords) at those points.

In essence, while the prevalence of quantized datasets makes finding a ready-made solution for non-quantized harmonization difficult, the technical means to build one do exist. The process involves three main steps: 1. **Parsing:** Using a library like `pretty_midi` or `music21` itself to read the raw MIDI file and create a data structure that preserves the exact timing of every note. 2. **Analysis:** Writing custom code to analyze the melodic stream and identify potential locations for chord changes based on musical criteria (phrase endings, melodic stability, etc.) rather than a rigid grid. 3. **Generation:** Applying algorithmic rules (e.g., diatonic triads, common progressions) to assign chords to the identified segments of the melody.

This approach ensures that the unique character of the input melody, including its expressive timing, is respected and becomes a driving force in the generation of the accompaniment. It transforms the perceived limitation of non-quantized input into a feature that allows for more nuanced and musically expressive harmonization.

Strategic Recommendations for Implementation

Given the absence of a single, recently released, off-the-shelf Python package that meets all the user's stringent requirements, the most effective strategy is not to continue searching for such a tool but to construct a custom harmonization pipeline using established, robust, and actively maintained libraries. This approach provides complete control over the algorithmic logic, guarantees compatibility with non-quantized input, and circumvents the limitations of ML-based systems. The following recommendations outline a clear path forward, prioritizing a balance between power, flexibility, and development effort.

Primary Recommendation: Develop a Custom Solution Using the `music21` Toolkit

The strongest recommendation is to leverage the `music21` library as the foundation for a bespoke harmonization tool. `music21` stands out because it uniquely combines three critical attributes identified during the analysis: active maintenance, a deep-rooted algorithmic philosophy, and unparalleled power and flexibility for music representation and manipulation [77] [181]. Its latest releases focus on improved typing and better MIDI support, indicating ongoing development into 2025 and beyond [181]. Most importantly, `music21` is not merely a MIDI file handler; it is a comprehensive toolkit for computational musicology that operates on the principles of music theory [77] [180]. This makes it the ideal engine for implementing a truly rule-based harmonization system.

The implementation path would involve the following steps:

1. **Parse the Melody:** Use `music21`'s `converter.parse()` function to load the input monophonic MIDI file. This will create a `Stream` object containing the melody's notes with their precise, non-quantized timing intact [159].
2. **Implement Harmonization Logic:** Create a separate Python module or function to house the algorithmic rules. This is the core of the custom solution. The logic could start with simple rules and evolve in complexity:

- **Key Detection:** First, determine the likely key of the melody. This can be done by analyzing the frequency of pitches or by allowing the user to specify the key.
- **Candidate Chords:** For each segment of the melody (e.g., a measure or a phrase), identify the diatonic triads (and potentially seventh chords) of the selected key. Any melody note falling within a chord's pitch class is a candidate for that chord [204].
- **Chord Progression:** Apply rules to select a sequence of chords. This could involve using a predefined list of common progressions (e.g., I-IV-V-I, vi-IV-I-V) or implementing a Markov chain based on common chord-to-chord transitions [49].
- **Voice Leading:** To improve quality, add rules for smooth voice leading. For example, when moving from one chord to the next, minimize the distance individual voices (the notes of the chords) travel [231].

1. **Construct the Accompaniment:** Once the chord progression is generated, construct the accompaniment tracks. This typically involves creating a bass line (often the root of each chord) and a block or broken chord texture for the upper voices. `music21`'s `stream.Stream` and `note.Note`/`chord.Chord` objects make it straightforward to assemble these parts.

2. **Output the Result:** Combine the original melody stream with the newly created accompaniment streams into a multi-track `midi.stream.Score` and export it as a new multi-track MIDI file using `music21`'s `write()` method [79].

The primary advantage of this approach is its fidelity to the user's request. It is purely algorithmic, respects the continuous-time nature of the input melody, and offers complete creative control over the harmonic output. The main disadvantage is the required development effort; it is not a "one-command" solution but rather a programming project. However, the power and extensibility gained are substantial.

Secondary Recommendation: Utilize Foundational MIDI Libraries for Data Handling

Regardless of the chosen harmonization logic, robust MIDI file I/O is essential. For the initial parsing and final writing stages, smaller, focused libraries can be highly effective. `pretty_midi` is an excellent choice for reading and analyzing MIDI files, offering clean abstractions over the raw MIDI data [97]. For writing the final accompaniment, a library like `MIDIUtil` is a lightweight, pure Python option that is easy to integrate [189]. Using these specialized tools alongside `music21` can streamline the overall pipeline. `music21` can handle the complex musical representation and logic, while `MIDIUtil` can efficiently produce the final `.mid` file.

Tertiary Recommendation: Explore Community Projects as Proof-of-Concept Code

Before committing to a full `music21` implementation, it can be beneficial to examine existing community efforts. The GitHub repository `wcyuan/Harmonize` contains a script named `harmonize.py` that explicitly aims to provide a list of possible chord harmonizations for a given melody [251]. Although its documentation is sparse, cloning this repository and studying its code could provide a valuable proof-of-concept or a simple template. It may contain basic implementations of mapping melody notes to candidate chords that can be adapted and extended within a more robust `music21`-based framework. This can significantly lower the initial barrier to entry and provide a working baseline for further development.

In conclusion, the optimal path forward is to abandon the search for a mythical, perfect package and embrace a "build-it-yourself" strategy. By combining the powerful, theory-rich `music21` library with a modular, rule-based harmonization engine and simple MIDI utilities, the user can create a highly effective, customizable, and future-proof tool that precisely matches their vision for an algorithmic, non-quantized MIDI harmonizer.

Reference

1. billyblu2000/AccoMontage2 https://github.com/billyblu2000/accomontage2
2. Best Vocal Harmonization Plugins 2026: Harmony Engine ... https://www.antarestech.com/blog/best-vocal-harmonization-plugins-2026?srsltid=AfmBOoqQRMhnIc2lFifxv0hLgi_5B_lzyxWUupqUUBeBJtUUtcgeOoYY
3. ACCOMONTAGE2: A COMPLETE HARMONIZATION AND ... https://archives.ismir.net/ismir2022/paper/000029.pdf
4. Python-Based Accordion Tuner https://www.accordionists.info/threads/python-based-accordion-tuner.16029/
5. AccoMontage2: A Complete Harmonization and ... https://ui.adsabs.harvard.edu/abs/arXiv:2209.00353
6. AccoMontage2: Harmonization System | PDF https://www.scribd.com/document/740578763/Accomontage2
7. 9 Best LLM Orchestration Tools for Enterprise in 2026 https://www.ayautomate.com/blog/best-llm-orchestration-tools
8. arXiv:2209.00353v1 [cs.SD] 1 Sep 2022 https://arxiv.org/pdf/2209.00353
9. Anticipatory Music Transformer https://arxiv.org/html/2306.08620v2
10. Notochord: a Flexible Probabilistic Model for Real-Time ... https://iil.is/pdf/2022_aimc_shepardson_et_al_notochord.pdf
11. Non-Standard Sound Synthesis with Dynamic Models https://pearl.plymouth.ac.uk/context/foahb-theses-other/article/1265/viewcontent/2013Valsamakis341008phd.pdf
12. Designing novel time-frequency scales for interactive ... https://theses.hal.science/tel-04237166/file/BAZIN_Theis_these_2023.pdf
13. Anticipatory Music Transformer https://johnthickstun.com/assets/pdf/anticipatory-music-transformer.pdf
14. MTU: A Melody Extraction Method Using Swin-Unet ... https://www.computer.org/csdl/magazine/mu/2026/01/11036666/27B98C5l5XW
15. Generative Music Composition Software Systems Using ... https://gupea.ub.gu.se/bitstreams/8682e1aa-cef0-472b-9d76-76b0204431fa/download
16. An Interactive System for Visualizing and Manipulating ... https://www.mtg.upf.edu/static/media/Eakin-Rich-Master-thesis-2009.pdf

17. Mathematical Foundations of Polyphonic Music Generation ... https://arxiv.org/pdf/2601.03612
18. A curated list of awesome Common Lisp ... https://github.com/codyreichert/awesome-cl
19. awesome-rainmana/README.md at master https://github.com/rainmana/awesome-rainmana/blob/master/README.md
20. for-open-source/data.json at main https://github.com/1Password/for-open-source/blob/main/data.json
21. GitHub · Change is constant. GitHub keeps you ahead. · GitHub https://github.com/
22. GitHub - srid/awesome-stars: My starred repos https://github.com/srid/awesome-stars
23. awesome-stars/README.md at master https://github.com/LumaKernel/awesome-stars/blob/master/README.md
24. Awesome-Java/README.md at main https://github.com/Hunterdii/Awesome-Java/blob/main/README.md
25. ACE-Step 1.5 Ultimate Guide (Must Read) https://github.com/ace-step/ACE-Step-1.5/blob/main/docs/en/Tutorial.md
26. dav0dea/goofi-pipe: real-time neuro-/biosignal processing ... https://github.com/PhilippThoelke/goofi-pipe
27. GenAI Agents: Comprehensive Repository ... https://github.com/nirdiamant/genai_agents
28. Generating chord progression from melody with flexible harmonic ... https://link.springer.com/article/10.1186/s13636-023-00314-6
29. [PDF] Decomposer: Learning to Decompile Symbolic Music to Programs - arXiv https://arxiv.org/pdf/2607.01849
30. Natural Language Processing Methods for Symbolic Music ... https://dl.acm.org/doi/full/10.1145/3714457
31. (PDF) Generating chord progression from melody with flexible harmonic ... https://www.researchgate.net/publication/377413795_Generating_chord_progression_from_melody_with_flexible_harmonic_rhythm_and_controllable_harmonic_density
32. Music and Audio Computing Lab | CT | KAIST https://mac.kaist.ac.kr/pubs.html
33. Artificial intelligence in music : a bibliometri… — Library of Science https://bibliotekanauki.pl/articles/64407309
34. POP909 Dataset: Annotated Chinese Pop Songs - Emergent Mind https://www.emergentmind.com/topics/pop909-dataset

35. AI-Enabled Text-to-Music Generation: A Comprehensive Review of ... - MDPI https://www.mdpi.com/2079-9292/14/6/1197
36. [PDF] Recent Advances in Music Generation: Methods, Evaluation, and ... - HAL https://hal.science/hal-05452282v1/document
37. [PDF] GENERATIVE ADVERSARIAL POST-TRAINING MITI - OpenReview https://openreview.net/pdf?id=FXm5U16vxD
38. cfogelklou/midi-mcp: Gives your AI agents the capability to ... https://github.com/cfogelklou/midi-mcp
39. GitHub - brightlikethelight/music21-mcp-server ... https://github.com/brightlikethelight/music21-mcp-server
40. awesome-python/README.md at main https://github.com/dylanhogg/awesome-python/blob/main/README.md
41. MoonWalker440/Music-Megathread https://github.com/MoonWalker440/Music-Megathread
42. TTS-arxiv-daily/README.md at master https://github.com/liutaocode/TTS-arxiv-daily/blob/master/README.md
43. Large Language Models'Internal Perception of Symbolic ... https://arxiv.org/html/2507.12808v1
44. Benchmarking Large Language Models on Symbolic Music ... https://arxiv.org/html/2509.23350v1
45. Large Language Models' Internal Perception of Symbolic ... https://arxiv.org/pdf/2507.12808
46. MusiChat: Vibe Composing for Music Creation https://arxiv.org/html/2607.24873v1
47. On the de-duplication of the Lakh MIDI dataset https://arxiv.org/pdf/2509.16662
48. A Survey on Deep Learning for Symbolic Music Generation https://dl.acm.org/doi/full/10.1145/3597493
49. SentHYMNent: An Interpretable and Sentiment-Driven ... https://dl.acm.org/doi/abs/10.1145/3637528.3671626
50. AI Harmonizer: Expanding Vocal Expression with a ... https://arxiv.org/html/2506.18143v1
51. An Algorithmic Approach to Automated Symbolic ... https://dl.acm.org/doi/abs/10.1145/3625135.3625137
52. Real-Time Pattern Recognition of Symbolic Monophonic ... https://dl.acm.org/doi/fullHtml/10.1145/3678299.3678329
53. MusicLang/maidi: Work with symbolic music gen AI easily ... https://github.com/MusicLang/maidi

54. Discover MIDI Dataset https://github.com/asigalov61/discovermidi
55. Metacreation-Lab/GigaMIDI-Dataset https://github.com/metacreation-lab/gigamidi-dataset
56. Melodfy : AI-Powered Piano Audio to MIDI Converter https://github.com/HemantKArya/Melodfy
57. harmonylang/harmony: A Python-like programming ... https://github.com/harmonylang/harmony
58. keymapperorg/KeyMapper: An Android app to remap ... https://github.com/keymapperorg/KeyMapper
59. wangrongding/github-university https://github.com/wangrongding/github-university
60. pybitcoin/pybitcoin/passphrases/english_words.py at master https://github.com/stacks-archive/pybitcoin/blob/master/pybitcoin/passphrases/english_words.py
61. understanding-npm/unpm-dat/data/names.json at master https://github.com/nodesource/understanding-npm/blob/master/unpm-dat/data/names.json
62. such/ignore.txt at main · yeerma/such https://github.com/yeerma/such/blob/main/ignore.txt
63. sfgfssfgsdgf/README.md at master https://github.com/tismalhas/sfgfssfgsdgf/blob/master/README.md
64. Generate a random password or randomized multiple-word ... https://gist.github.com/pmarreck/28b3049a1a70b8b4f2eaff4466d0c76a?permalink_comment_id=5995395
65. The ultimate guide to making melodies WITHOUT music theory https://www.youtube.com/watch?v=bZ6Xz7zAMAk
66. Melody Extraction https://purl.pt/282/1/v3d2/projects/midilib/midilib/english/skydemo.html
67. How to improve your melodies with rhythmic variations (FREE ... https://www.youtube.com/watch?v=Tm8VSjwjsss
68. Automated Arrangements of Multi-Part Music for Sets ... https://arxiv.org/abs/2301.12084
69. Melody Extraction https://www.justinsalamon.com/melody-extraction.html
70. Create Instant Melodic Propulsion in Your Scores https://www.youtube.com/watch?v=zwvHPSYRSYw&vl=en-US
71. Algorithmic Composition https://www.musicprocessing.net/features/algorithmic-composition/
72. THESE Music Theory HACKS will 10X Your Melodies https://www.youtube.com/watch?v=sMbR7GmVA2A

73. An Approach To Composing Melody, Part 1 https://www.youtube.com/watch?v=jrgpuxtZUu4
74. Make Your Comping Melodic! (7 Ways) https://www.youtube.com/watch?v=AN2tVoSBU2A
75. music21 Documentation — music21 Documentation https://music21.org/
76. music21 --> stream.chordify() not working https://stackoverflow.com/questions/33663528/music21-stream-chordify-not-working
77. music21 https://pypi.org/project/music21/
78. music21 - R-Music https://r-music.rbind.io/posts/2018-10-06-music21/
79. Please include the Python package music21 for musical ... https://community.openai.com/t/please-include-the-python-package-music21-for-musical-projects/1228290
80. rdmgator12/awesome-claude-connectors https://github.com/rdmgator12/awesome-claude-connectors
81. Chatbot Arena Query Topic Modelling https://gist.github.com/shekharx/e1c36ed62b119fd30e739ee8bc589468
82. GitHub - zhtjtcz/Mine-Arxiv https://github.com/zhtjtcz/Mine-Arxiv
83. GitHub - HackerX7889/Fortran-Code-Repos https://github.com/HackerX7889/Fortran-Code-Repos
84. data_filtering_[process]valid_ast.ipynb https://github.com/PELAB-LiU/jupyter_nbs_empirical/blob/master/data_filtering_%5Bprocess%5Dvalid_ast.ipynb
85. AI chord and melody generators: Yea or Nay? https://www.facebook.com/groups/emproducers/posts/1528990097751614/
86. Chord-Transformer:Chord-Progression Guided ... https://openreview.net/forum?id=A7jXhEXORd
87. I make TRAP MELODY LOOPS for MUSIC PRODUCERS ... https://www.instagram.com/reel/DZaG5sopn78/
88. Comment and we'll send you a free download for MIDI ... https://www.instagram.com/p/DYpj9MmxqFM/?__d=1%E9%A3%9E%E6%9C%BA%E5%B0%8F%E5%8F%B7%E5%93%AA%E9%87%8C%E8%B4%AD%E4%B9%B0%E2%9C%94%EF%B8%8F%2Baixiaohao.com%2B%E2%AD%90%2B%2B%E7%B2%BE%E5%85%BB%E6%94%AF%E4%BB%98%E5%AE%9D%E5%8F%B7%E5%A6%82%E4%BD%95%E5%90%B8%E7%B2%89
89. NEVER STARE AT A BLANK PIANO ROLL AGAIN. DESTROY ... https://www.tiktok.com/@frddyyy.mvp/video/7645300707674836246
90. day 21/30 sharing a free plugin Today, a unique instrument ... https://www.instagram.com/p/DYU7_4tPy_7/?

__d=1%E7%A7%81%E5%AF%86%E6%9F%A5%E8%AF%A2%E8%BE%B9%E6%8E%A7%E6%83%85%E5%86%B5%2C%E8%B6%85%E8%B5%9E%E7%9A%84%E7%BB%88%E7%AB%AF%E7%A4%BE%E5%B7%A5%E5%BA%93%E6%89%BE%E4%BA%BA%E2%9C%94%EF%B8%8Fzhaoren.cyou

91. Video di frddyyy. (@frddyyy.mvp) con suono originale | Trap ... https://www.tiktok.com/@frddyyy.mvp/video/7646462024788102422
92. Trapped Kolter Chords https://www.tiktok.com/discover/trapped-kolter-chords
93. Everyone's arguing about whether AI music is “real ... https://www.facebook.com/trey.alteredent/posts/everyones-arguing-about-whether-ai-music-is-realmeanwhile-theyre-missing-the-big/10242620220391935/
94. Pattern in Music https://s3-euw1-ap-pe-df-pch-content-store-p.s3.eu-west-1.amazonaws.com/9781003457541/ae1959ba-27ab-48f8-af8f-d4c1d623b307/chapters/chapter8.pdf?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAQFVOSJ57TJK3L5FC%2F20260806%2Feu-west-1%2Fs3%2Faws4_request&X-Amz-Date=20260806T184218Z&X-Amz-Expires=172800&X-Amz-Signature=5602389b1876c6b273300aa683015d8c615c38fb7be8dc19717ec2abd8bc1a12&X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3D%2210.4324_9781003457541-9_chapterpdf.pdf%22
95. MIDIUtil https://pypi.org/project/MIDIUtil/
96. midiharmony https://pypi.org/project/midiharmony/
97. pretty-midi https://pypi.org/project/pretty-midi/
98. REMI-z https://pypi.org/project/REMI-z/
99. sreyasaju/harmonize https://github.com/sreyasaju/harmonize
100. install midi, midi_maniulation on Linux for python3 https://stackoverflow.com/questions/50028697/install-midi-midi-maniulation-on-linux-for-python3
101. musiclang https://pypi.org/project/musiclang/
102. Programming with MIDI in Python | Getting started and sending ... https://www.youtube.com/watch?v=JYslZkc90GI
103. pretty-midi-fix https://pypistats.org/packages/pretty-midi-fix
104. A Survey on Deep Learning for Symbolic Music Generation https://dl.acm.org/doi/10.1145/3597493
105. Natural Language Processing Methods for Symbolic Music ... https://arxiv.org/pdf/2402.17467
106. Natural Language Processing Methods for Symbolic Music ... https://dl.acm.org/doi/abs/10.1145/3714457

107. Foundation Models for Music: A Survey https://arxiv.org/html/2408.14340v1
108. arXiv:2402.16153v1 [cs.SD] 25 Feb 2024 https://arxiv.org/pdf/2402.16153
109. Suitable Music Generation from Videos using an Affective ... https://arxiv.org/html/2311.00968v2
110. Path-dependent program induction under resource ... https://arxiv.org/html/2606.20623
111. From Audio Deepfake Detection to AI-Generated Music ... https://arxiv.org/pdf/2412.00571
112. pycomposer https://pypi.org/project/pycomposer/
113. https://pypi.org/33.sitemap.xml https://pypi.org/33.sitemap.xml
114. Algorithmic midi sequencer for music creation https://www.facebook.com/groups/AlgoComp/posts/8257830080915707/
115. Built a music theory tool that analyzes your MIDI files and ... https://www.reddit.com/r/ableton/comments/1pae4ji/built_a_music_theory_tool_that_analyzes_your_midi/
116. Matchmaker: An Open-Source Library for Real-Time Piano ... https://arxiv.org/html/2510.10087v1
117. Live Coding Melody and Harmony in JavaScript https://dl.acm.org/doi/fullHtml/10.1145/3678299.3678336
118. Deep gradient reinforcement learning for music improvisation ... https://pmc.ncbi.nlm.nih.gov/articles/PMC11784531/
119. Top Python Libraries of 2025 https://www.edge-ai-vision.com/2026/01/top-python-libraries-of-2025/
120. Write simple melodies with Music21 and Python - YouTube https://www.youtube.com/watch?v=MYVWPmzsRz8
121. Chord Rules for Jazz? - Page 2 - MusicDevelopments Forum https://www.kvraudio.com/forum/viewtopic.php?t=616766&start=15
122. Module Reference - music21 Documentation https://music21.org/music21docs/moduleReference/index.html
123. How to create and analyse chords in Music21 - YouTube https://www.youtube.com/watch?v=qy5fc5lJ6U0
124. AccoMontage2.ipynb - Google Colab https://colab.research.google.com/github/billyblu2000/AccoMontage2/blob/master/colab-notebooks/AccoMontage2.ipynb
125. How would you take a melody and harmonize it for jazz? - Reddit https://www.reddit.com/r/musictheory/comments/14yfm7/how_would_you_take_a_melody_and_harmonize_it_for/

126. How to Harmonize Any Melody - YouTube https://www.youtube.com/watch?v=VR1dzkCjhgY
127. A Complete Harmonization and Accompaniment Arrangement System https://arxiv.org/abs/2209.00353
128. GitHub - zhaojw1998/AccoMontage: Code and MIDI demo for paper https://github.com/zhaojw1998/AccoMontage
129. How To Create Melodies Without A MIDI Controller OR Music ... https://www.youtube.com/watch?v=LcZsIKAseio
130. HOW TO MAKE MELODIES - Without Music Theory https://www.youtube.com/watch?v=XYXyxT2C-uQ
131. Harmony Releases https://harmonydata.ac.uk/releases/
132. Publications - PIANO https://piano-project.it/publications.html
133. Publications https://nickhwang.com/publications/
134. OpenUtau USTx Harmonies Generator GUI · ... https://github.com/formergen/USTxHarmsGenGUI
135. Simple Index https://mirrors.sustech.edu.cn/pypi/simple/
136. Natural Language Processing Methods for Symbolic Music ... https://hal.science/hal-04621444v1/document
137. Symbolic music structure analysis with graph ... https://www.sciencedirect.com/science/article/pii/S0925231225028802
138. Can people suggest ways of encoding polyphonic music ... https://www.facebook.com/groups/876194196241093/posts/1680223245838180/
139. Real-time Symbolic Music Generation using Deep ... https://aaltodoc.aalto.fi/bitstreams/8f4c931b-4dd3-4d47-a711-c16bb1689490/download
140. Measure-Based Automatic Music Composition with Modern ... https://transactions.ismir.net/articles/10.5334/tismir.163
141. Mel2Word: A Text-Based Melody Representation for ... https://journals.sagepub.com/doi/10.1177/20592043231216254
142. arXiv:2311.00968v2 [cs.SD] 4 Mar 2024 - Dorien Herremans https://dorienherremans.com/sites/default/files/2311.00968.pdf
143. Algorithmic Music Composition With Python https://www.youtube.com/watch?v=37S0ZDpJpc4
144. Algorithmic Music Generation with Python https://www.reddit.com/r/Python/comments/1ge68i5/algorithmic_music_generation_with_python/
145. Archive https://dadmaan.com/archive/

146. Publications | AI in Computational Arts, Music, and Games https://aicomparts.com/publications/
147. music-x-lab/midi-chord-recognition https://github.com/music-x-lab/midi-chord-recognition
148. Melodia is a Python-based tool for generating musical ... https://github.com/caseybarajas/Melodia
149. MIDI to Virtual Piano Converter https://github.com/Stereo101/python-MidiToVirtualPianoMacro
150. Music2MIDI: Pop Music to MIDI Piano Cover Generation https://github.com/ytinyui/music2midi
151. MIDI, those cheesy sounds from the 90s? Wrong! Symbolic ... https://www.youtube.com/watch?v=x0mfWfxLCkI
152. MIDI - Symbolic Music Analysis and Generation with Python https://www.classcentral.com/course/youtube-midi-those-cheesy-sounds-from-the-90s-wrong-symbolic-music-vs-python-mateusz-modrzejewski-495634
153. How to generate music with Python: The Basics https://medium.com/@stevehiehn/how-to-generate-music-with-python-the-basics-62e8ea9b99a5
154. Contrastive Audio-MIDI Learning for Symbolic-domain ... https://eusipco2025.org/wp-content/uploads/pdfs/0000426.pdf
155. Best python library to work with MIDI files? : r/MLQuestions https://www.reddit.com/r/MLQuestions/comments/c9j22a/best_python_library_to_work_with_midi_files/
156. Create MIDI Files Using Python & AI - No Coding Skills ... https://www.youtube.com/watch?v=1AOIgXOqHKA
157. arXiv:2106.05630v1 [cs.SD] 10 Jun 2021 https://arxiv.org/pdf/2106.05630
158. music21.stream.base https://music21.org/music21docs/moduleReference/moduleStreamBase.html
159. User's Guide, Chapter 1: Installing and Getting Started with https://music21.org/music21docs/usersGuide/usersGuide_01_installing.html
160. Music21: Composing with the Harmonic Series and Python https://www.youtube.com/watch?v=XmsF9HmIEis
161. Building Documentation https://music21.org/music21docs/developerReference/buildingDocumentation.html
162. music21 seems to modify MIDI file when reading/rewritting it https://stackoverflow.com/questions/77367782/music21-seems-to-modify-midi-file-when-reading-rewritting-it
163. mingus 0.5.1 documentation https://bspaans.github.io/python-mingus/

164. mididings https://pypi.org/project/mididings/
165. Melodic development via imitation https://www.youtube.com/watch?v=G09imOGFilc
166. snisim/pytakt: A Python library for music description, ... https://github.com/snisim/pytakt
167. Tool recommendations - Python Packaging User Guide https://packaging.python.org/guides/tool-recommendations/
168. ZhikangNiu/arxiv_daily https://github.com/ZhikangNiu/arxiv_daily
169. Let me post this, blow your mind, and run away, but please, ... https://www.facebook.com/groups/puredata/posts/10162719399639495/
170. Before synthesisers soundtracked generations of electric ... https://www.facebook.com/nfsaa/posts/before-synthesisers-soundtracked-generations-of-electric-dreams-many-questioned-/1509647004537932/
171. Kandiskyscore 1.0.7(Octobre 2025) -- Tempo automation ... https://www.facebook.com/groups/160548580661321/posts/24800686419554195/
172. For the past months I've been working almost relentlessly ... https://www.instagram.com/p/DPDrP3RiFL5/
173. Start your musical journey at The Piano Workshop at ... https://www.facebook.com/61554980410612/posts/in-a-world-of-algorithms-make-something-humanstart-your-musical-journey-and-crea/1222767663741660 13/
174. A survey of deep learning audio generation methods https://arxiv.org/pdf/2406.00146
175. Which AI model is used for music generation? https://www.facebook.com/groups/390522607205963/posts/618028011122087/
176. OSAQ is built around a gently swung clock from Pamela's ... https://www.facebook.com/groups/eurorackmodular/posts/2365669970541572/
177. github-university/README.md at main https://github.com/wangrongding/github-university/blob/main/README.md
178. music https://pypi.org/project/music/
179. Lib https://github.com/python/cpython/tree/main/Lib
180. Introduction to Music21 for Python https://www.youtube.com/watch?v=zanx4IS3p24
181. Releases · cuthbertLab/music21 https://github.com/cuthbertLab/music21/releases
182. Video 1a: music21 Coding Tutorial for OCW Learners https://www.youtube.com/watch?v=BZwdYmPS5f8

183. A Python repository that creates MIDI files in an un ... http://forum.pdpatchrepo.info/topic/15786/a-python-repository-that-creates-midi-files-in-an-un-conventional-way

184. Does anyone know of a program or site where I can ... https://www.reddit.com/r/midi/comments/1qyjr9b/does_anyone_know_of_a_program_or_site_where_i_can/

185. Python Music Production #3 - Python Libraries for Music https://www.youtube.com/watch?v=4Nz-jVQljfY

186. Audio to midi using python https://www.youtube.com/watch?v=5_cJUoQuwlY

187. Music and Musicology | Python across all Disciplines https://docs.pyclubs.org/python-across-all-disciplines/disciplines/music-and-musicology

188. Simple, easy to use MIDI extraction module/library for ... https://www.reddit.com/r/learnpython/comments/1rgax2o/simple_easy_to_use_midi_extraction_modulelibrary/

189. TheLongRunSmoke/midiutil: It's a pure Python MIDI library. https://github.com/TheLongRunSmoke/midiutil

190. Generate Music Through Python – A Complete Guide https://www.albinsblog.com/2025/01/generate-music-through-python-complete-guide.html

191. Installing HarmonyPy — harmony-py 0.0.1 documentation https://harmony-py.readthedocs.io/en/stable/user/install.html

192. Chapter 0 · Setup and Your First Sound - Subsequence https://subsequence.live/cookbook/00-setup.html

193. homeassistant | Prutser .. https://www.henriaanstoot.nl/tag/homeassistant/

194. midimelody https://pypi.org/project/midimelody/

195. pm-python https://pypi.org/project/pm-python/

196. ejfox/hand-midi-controller: Professional hand tracking MIDI ... https://github.com/ejfox/hand-midi-controller

197. MIDI Visualization - Free and Open Web based alternatives to ... https://www.youtube.com/watch?v=8RP5E_NWQOw

198. LMMS | Open Source Alternative to FL Studio, Ableton https://www.opensourcealternative.to/project/lmms

199. research - - Austin Oting Har https://www.austinotinghar.com/research/

200. samples https://github.com/microsoft/MIDI/tree/main/samples

201. RAWL Release Harmonic DNA Reconstruction + A FREE ... https://www.facebook.com/groups/viapgroup/posts/10165021967916613/

202. About the update coming later today https://www.facebook.com/groups/1718657492905695/posts/1724393502332094/

203. Algorithmic Composition of Modern Classical Music : r/Python https://www.reddit.com/r/Python/comments/1e6zh24/dodecaphony_algorithmic_composition_of_modern/
204. Midi Music Theory 49 - Diatonic Chord Progressions https://www.youtube.com/watch?v=su9wH1Yonr0
205. Niko's MIDI Pack - FREE Chord Progression Guide & MIDI ... https://www.youtube.com/watch?v=0j_AoM717Eo
206. Harmonic Progressions https://www.teoria.com/en/exercises/hp.php
207. Music Theory: Chord Progression Types https://www.youtube.com/watch?v=eWalydQ834Q
208. Turn Any Melody Into Chord Progressions | Music Theory ... https://www.youtube.com/watch?v=AOcHSzsJj30
209. 12 MUST KNOW Chord Progressions All Musicians Can Use https://www.youtube.com/watch?v=a83prmdSTQk
210. The Most Important Chord Progression - Music Theory Crash ... https://www.youtube.com/watch?v=wtkYFQi8GpM
211. Hidden patterns of chord progressions https://www.youtube.com/watch?v=mBD5oJYMU0o
212. Jazzy Chord Progressions Super Varied Jazz Midi Chords ... https://www.youtube.com/watch?v=zOdYPq8bPuo
213. YouTube https://www.youtube.com/shorts/bPtOYY-uEdY
214. E01 Intro to Midi and Python https://www.youtube.com/watch?v=JxMKsPPEDAc
215. Automating MIDI Generation with Python https://dev.to/dm8ry/automating-midi-generation-with-python-a-comprehensive-guide-ckb
216. Hmbown/Bachbot https://github.com/Hmbown/Bachbot
217. awesome-stars/README.md at master https://github.com/JKirchartz/awesome-stars/blob/master/README.md
218. Throughput Database Github Keywords https://gist.github.com/SimonGoring/a89f5103dcb1b518a978709c457bf037
219. arXiv_daily/daily_papers/20260106_Tue/audio.md at main https://github.com/advanced-cs/arXiv_daily/blob/main/daily_papers/20260106_Tue/audio.md
220. 1024 PROMPTS · GitHub https://gist.github.com/VictorTaelin/8a8455e15b9c38c9177cc243e22e047f
221. Can you recommend a (preferably free) program that ... https://www.reddit.com/r/midi/comments/1enpgzl/can_you_recommend_a_preferably_free_program_that/

222. audio-development-tools/README.md at main https://github.com/Yuan-ManX/audio-development-tools/blob/main/README.md
223. sildater/parangonar: alignment and synchronization of music https://github.com/sildater/parangonar
224. Music Intelligence Lab https://github.com/Music-Intelligence-Lab
225. apasagic/audio-classification https://github.com/apasagic/audio-classification
226. QwenAudio/FunMusic: A fundamental toolkit designed for ... https://github.com/QwenAudio/FunMusic
227. Releases · ossia/score https://github.com/ossia/score/releases
228. Using a rotary encoder as a volume control https://gist.github.com/savetheclocktower/9b5f67c20f6c04e65ed88f2e594d43c1
229. Welcome to MiniDexed Discussions! #14 https://github.com/probonopd/MiniDexed/discussions/14
230. tubone24/midi-agent-skill https://github.com/tubone24/midi-agent-skill
231. CLAUDE.md - calvinw/fl-studio-mcp https://github.com/calvinw/fl-studio-mcp/blob/main/CLAUDE.md
232. MIDI-GPT: A Controllable Generative Model for Computer- ... https://ojs.aaai.org/index.php/AAAI/article/view/32138/34293
233. BERT-like Pre-training for Symbolic Piano Music ... https://arxiv.org/pdf/2107.05223
234. Real-Time Pattern Recognition of Symbolic Monophonic Music https://www.lucaturchet.it/PUBLIC_DOWNLOADS/publications/conferences/Real-time_pattern_recognition_of_symbolic_monophonic_music.pdf
235. Mamba-Diffusion Model with Learnable Wavelet for ... https://arxiv.org/pdf/2505.03314
236. Symbolic music structure analysis with graph ... https://zaguan.unizar.es/record/165170/files/texto_completo.pdf
237. arXiv:2410.02060v1 [cs.SD] 2 Oct 2024 https://arxiv.org/pdf/2410.02060
238. midi · GitHub Topics https://github.com/topics/midi
239. An Agent-Based Framework for Automated Higher-Voice ... https://arxiv.org/html/2509.24463v1
240. The Modelling of Structure in Symbolic Music Generation https://arxiv.org/html/2403.07995v1
241. Abstract https://arxiv.org/html/2002.09748v1
242. NotaGen: Advancing Musicality in Symbolic ... https://arxiv.org/html/2502.18008v2

243. MusicAOG: an Energy-Based Model for Learning and ... https://arxiv.org/html/2401.02678v1

244. Understanding and Generating Music Intrinsically with LLM https://arxiv.org/html/2402.16153v1

245. Symbolic Music Generation with Fine-grained Interactive ... https://arxiv.org/html/2410.08435v1

246. Algorithmic Composition: A Gentle Introduction to Music ... https://quod.lib.umich.edu/cgi/t/text/idx/s/spobooks/bbv9810.0001.001/1:19/--algorithmic-composition-a-gentle-introduction-to-music?rgn=div1;view=fulltext

247. Music Software - Karlheinz Essl https://www.essl.at/software.html

248. Algorithmic Composition: A Gentle Introduction to Music ... https://quod.lib.umich.edu/s/spobooks/bbv9810.0001.001/--algorithmic-composition-a-gentle-introduction-to-music

249. API documentation - MIDI CC and NRPN database https://api.midi.guide/docs/

250. slowkow/harmonypy - GitHub https://github.com/slowkow/harmonypy

251. Harmonize/harmonize.py at master · wcyuan/Harmonize - GitHub https://github.com/wcyuan/Harmonize/blob/master/harmonize.py

252. How to generate Euclidean Rhythms with Python and Music21 https://www.youtube.com/watch?v=y1EixzVEJlo

253. Call for Papers | ISMIR 2026 | Abu Dhabi, UAE https://ismir2026.ismir.net/authors/call-for-papers

254. ISMIR 2026 Submission 363 https://anonymous55aht.github.io/

255. Tutorials | ISMIR 2026 | Abu Dhabi, UAE https://ismir2026.ismir.net/tutorials

256. Transactions of the International Society for Music ... - ISMIR https://transactions.ismir.net/

257. Recent advances in music generation: methods, evaluation ... https://link.springer.com/article/10.1007/s10462-026-11582-x

258. [2509.01588] From Discord to Harmony https://arxiv.org/abs/2509.01588

259. Call for Tutorials | ISMIR 2026 | Abu Dhabi, UAE https://ismir2026.ismir.net/call-for-tutorials

260. mir-datasets.md https://github.com/ismir/mir-datasets/blob/master/outputs/mir-datasets.md

261. This MIDI Visualizer Understands Jazz Harmony ... https://www.youtube.com/watch?v=BdCtpqQHSOg

262. Harmonize Melody 6 Different Ways - No Chord Progression ... https://www.youtube.com/watch?v=NtENn_j5cR4

263. Prism Midi Sequencer - Tutorial 10: Create Gorgeous ... https://www.youtube.com/watch?v=4sHvc2VhsRc

264. 4 Ways to Reharmonize Any Song (Tim Hates #3) https://www.youtube.com/watch?v=KPjkq5B_vZs

265. How to Harmonize on Piano: Simple Patterns for Any Melody https://www.youtube.com/watch?v=pUm8DMOhKOw

266. Piano Improvisation Using MAPPING TONAL HARMONY Jazz ... https://www.youtube.com/watch?v=-GlGwBNEb-Y

267. Melody Harmonizer by Bitsonic - Harmony Generator ... https://www.kvraudio.com/product/melody-harmonizer-by-bitsonic

268. 12-Key Workouts: The Ultimate Guide to Tonal Harmony ... https://www.youtube.com/watch?v=EBWR9ijfwGY

269. A collection of awesome custom nodes for ComfyUI https://github.com/ComfyUI-Workflow/awesome-comfyui

270. TanKaiYi0220/Online-Judge-Problems https://github.com/TanKaiYi0220/Online-Judge-Problems

271. Sounddevice in python, time vector of input and output ... https://stackoverflow.com/questions/78869667/sounddevice-in-python-time-vector-of-input-and-output-signal-synchronized

272. Which Chords Should I Play Over This Melody? https://www.youtube.com/watch?v=gPi6Tyyw-Hc

273. Harmonizing Melody 4 | conforming to changing Modes https://www.youtube.com/watch?v=6norh6eBkhA

274. How to sing Stuck with U like Ariana Grande & Justin Bieber in ... https://www.youtube.com/watch?v=UnJQdBe3A-Y

275. Analysis and Visualization of Musical Structure using ... https://arxiv.org/html/2404.15208v1

276. Updated 2026-08-18 · GitHub https://gist.github.com/masta-g3/8f7227397b1053b42e727bbd6abf1d2e

277. MIDI-VALLE: Piano Performance Synthesis with VALLE-E https://github.com/nii-yamagishilab/MIDI-VALLE

278. juzzlin/Noteahead: A MIDI tracker and sequencer with built- ... https://github.com/juzzlin/Noteahead

279. The GigaMIDI Dataset with Features for Expressive Music ... https://arxiv.org/html/2502.17726v1

280. Quantizing MIDI: You Might Be Doing This WRONG https://www.youtube.com/watch?v=jDOb_6txank

281. 5 MIDI Quantization Tips https://midi.org/5-midi-quantization-tips
282. Interval & Random Harmony Generator Script for MIDI playing ... https://www.youtube.com/watch?v=QeTsmlNEF7I
283. The Difference between Quantized and Unquantized MIDI https://klang.io/help/quantized-unquantized-midi-difference/
284. Realtime Midi Quantize - Feature Requests https://community.blokas.io/t/realtime-midi-quantize/2006
285. 4 Bars Monophonic Melodies Dataset (Pitch Sequence) https://zenodo.org/records/13369389
286. asigalov61/Tegridy-MIDI-Dataset https://github.com/asigalov61/Tegridy-MIDI-Dataset
287. Quantization in MIDI Production https://motifkit.com/quantization/
288. (PDF) The GigaMIDI Dataset with Features for Expressive ... https://www.researchgate.net/publication/389351831_The_GigaMIDI_Dataset_with_Features_for_Expressive_Music_Performance_Detection